

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2-3

Выполнил:

Шаповалов Сергей

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на документ, сформированный в рамках ДЗ4, реализуйте разделение вашего бэкенд-приложения на микросервисы.

реализовать Dockerfile для каждого сервиса;
написать общий docker-compose.yml;
настроить сетевое взаимодействие между сервисами.

подключить и настроить rabbitMQ/kafka;

реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

До этого проект был монолитом: один бекенд, одна кодовая база, одна база данных и единая точка исполнения. В таком варианте все модули (авторизация, пользователи, рестораны, бронирования, отзывы) живут в одном приложении и зависят от одной общей схемы данных.

После разбиения проект стал микросервисной системой, где:

- `auth-service` отвечает только за регистрацию, логин и JWT;
- `users-service` отвечает только за данные пользователя;
- `restaurants-service` отвечает за рестораны, меню, фото и кухни;
- `reservations-service` отвечает за бронирования;
- `reviews-service` отвечает за отзывы;
- `http-logger` служит для логирования HTTP событий;
- каждый сервис имеет свою собственную PostgreSQL базу данных;
- все сервисы находятся в одной Docker-сети и общаются через RabbitMQ;
- HTTP-запросы и события идут не напрямую между сервисами, а через шину сообщений.

Что именно было вынесено

- `auth` был вынесен в отдельный сервис, чтобы авторизация и JWT не зависели от остальных модулей;
- `users` стал отдельным сервисом для профилей и пользовательских данных;
- `restaurants` стал независимым сервисом, который хранит данные о ресторанах, меню и фотографиях;
- `reservations` стал отдельным сервисом с жесткой привязкой к пользователю и ресторану;
- `reviews` стал отдельным сервисом для обработки отзывов;
- логирование HTTP запросов вынесено в отдельный `http-logger`.

Это соответствует принципу domain-driven decomposition: каждый сервис отвечает за свой домен и хранит данные только для него.

Каждый микросервис имеет свой Dockerfile. Рассмотрим пример из `auth-service`:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3001

CMD ["npm", "run", "dev"]
```

FROM node:20-alpine`

Это базовый образ. Он берет готовый контейнер с Node.js 20 и Alpine Linux. Alpine используется для маленького размера образа и быстрой сборки.

```
`WORKDIR /app`
```

Устанавливает рабочую директорию внутри контейнера. Все дальнейшие команды выполняются в `/app`.

```
`COPY package*.json ./`
```

Копирует в контейнер файлы `package.json` и `package-lock.json` (если есть). Это нужно, чтобы сначала установить зависимости, а не копировать весь проект целиком.

```
`RUN npm install`
```

Устанавливает зависимости проекта внутри контейнера. Из-за этого сервис получает все пакеты, которые нужны для запуска (Express, PostgreSQL client, JWT, RabbitMQ client и т.д.).

```
`COPY . .`
```

Копирует весь исходный код сервиса в контейнер: `src`, конфиги, JSON-описания API и прочие файлы.

```
`EXPOSE 3001`
```

Указывает, что сервис слушает порт 3001. Это документирование порта для удобства и внутренней структуры Docker, но сам порт реально пробрасывается через `docker-compose.yml`.

```
`CMD ["npm", "run", "dev"]`
```

Запускает приложение при старте контейнера. В этом проекте используется режим разработки (`npm run dev`), то есть приложение стартует через ts-node-dev/Node dev-сервер и автоматически реагирует на изменения в коде.

Почему нужны Dockerfile у каждого сервиса

- каждый сервис имеет собственные зависимости;
- у каждого можно отдельно задавать порт и окружение;
- можно собирать и запускать их независимо;

- это дает возможность запускать весь стек целиком через один ``docker-compose.yml``.

Как работает RabbitMQ в этом проекте

RabbitMQ — это брокер сообщений. Он принимает сообщения от производителей и отправляет их подписчикам.

В проекте используется два основных обменника:

``http.requests``

- каждый сервис, получив запрос, публикует JSON в него;
- ``http-logger`` подписан на этот exchange;
- он получает все HTTP-события и логирует их.

``app.events``

- сервисы публикуют события по типам, например ``user.registered`` или ``reservation.created``;
- подписчики подписаны на темы и получают только те события, которые им нужны.

Пример:

```
ch.publish("app.events", "user.registered", Buffer.from(msg));
```

Здесь:

- exchange: ``app.events``
- routing key: ``user.registered``
- подписчик может использовать ``#`` или конкретный ключ для получения нужных событий.

Почему RabbitMQ полезен

- декомпозирует зависимость между сервисами;
- уменьшает плотную связанность;
- позволяет вести логирование и события независимо;
- позволяет расширять функциональность без переписывания всего приложения.

Важный момент

RabbitMQ не заменяет HTTP. Он дополняет архитектуру:

- HTTP — для синхронных пользовательских вызовов;
- RabbitMQ — для асинхронных событий и интеграций.

Вывод:

В результате монолит был разбит на несколько независимых сервисов.

Каждый сервис:

- имеет свою ответственность;
- хранит свои данные в отдельной БД;
- общается через HTTP и RabbitMQ;
- запускается через Docker;
- работает в общей сети и при этом не зависит напрямую от остальных.

Это уже соответствует базовым принципам микросервисной архитектуры: слабая связанность, высокая автономность и масштабируемость.